AhamX Bodhi

Where I Becomes Infinite

Scaled Dot-Product Attention

Module 3.2: Transformer Architecture Deep-Dive

Prof. Sashikumaar Ganesan | IISc Bengaluru

Session Overview

What You Will Learn

- The step-by-step mechanics of the attention formula
- Why scaling by $\sqrt{d_k}$ is essential
- The role of the softmax in producing attention weights
- Causal masking for autoregressive generation
- Computational complexity and practical limits

Concept at a Glance

- **Duration:** 10 minutes
- **Bloom's Level:** B3 – Apply
- **Difficulty:** L3 – Advanced Intermediate
- **Module:** 3.2 Transformer Deep-Dive
- **Prerequisite:** Attention Is All You Need

Learning Objectives

By the End of This Session You Will Be Able To

1. **Write** the scaled dot-product attention formula from memory
2. **Explain** why the scaling factor $1/\sqrt{d_k}$ prevents vanishing gradients
3. **Trace** a worked numerical example through the full attention computation
4. **Distinguish** self-attention from cross-attention in terms of the Q, K, V sources
5. **Identify** where causal masking is applied and why it is needed for generation

Prerequisites and Connections

You Should Already Know

- Matrix multiplication and transpose
- Softmax function and probability distributions
- Dot product as a similarity measure
- What Q, K, V represent conceptually

This Concept Unlocks

- Multi-Head Attention (parallelising heads)
- Flash Attention (efficient implementation)
- KV-Cache (inference optimisation)
- Sparse and Linear Attention variants

The Scaled Dot-Product Attention Formula

The Core Equation

The complete formula for scaled dot-product attention is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

where $Q \in \mathbb{R}^{n \times d_k}$ (queries), $K \in \mathbb{R}^{m \times d_k}$ (keys), $V \in \mathbb{R}^{m \times d_v}$ (values), n is the query sequence length, m is the key/value sequence length, d_k is the key dimension, and d_v is the value dimension.

Step-by-Step Computation

Four Operations in Sequence

1. **Compute raw scores:** $S = QK^\top$, shape $n \times m$. Entry S_{ij} measures how much query i aligns with key j via dot product
2. **Scale:** Divide S by $\sqrt{d_k}$. Prevents large dot products from pushing softmax into saturation regions
3. **(Optional) Mask:** Add $-\infty$ to positions that should not be attended to (future tokens in causal attention, or padding positions)
4. **Softmax:** Apply row-wise softmax to produce attention weights A, shape $n \times m$. Each row sums to 1.0 and represents a distribution over positions
5. **Aggregate:** Multiply attention weights by values: Output $= AV$, shape $n \times d_v$. Each output is a weighted sum of value vectors

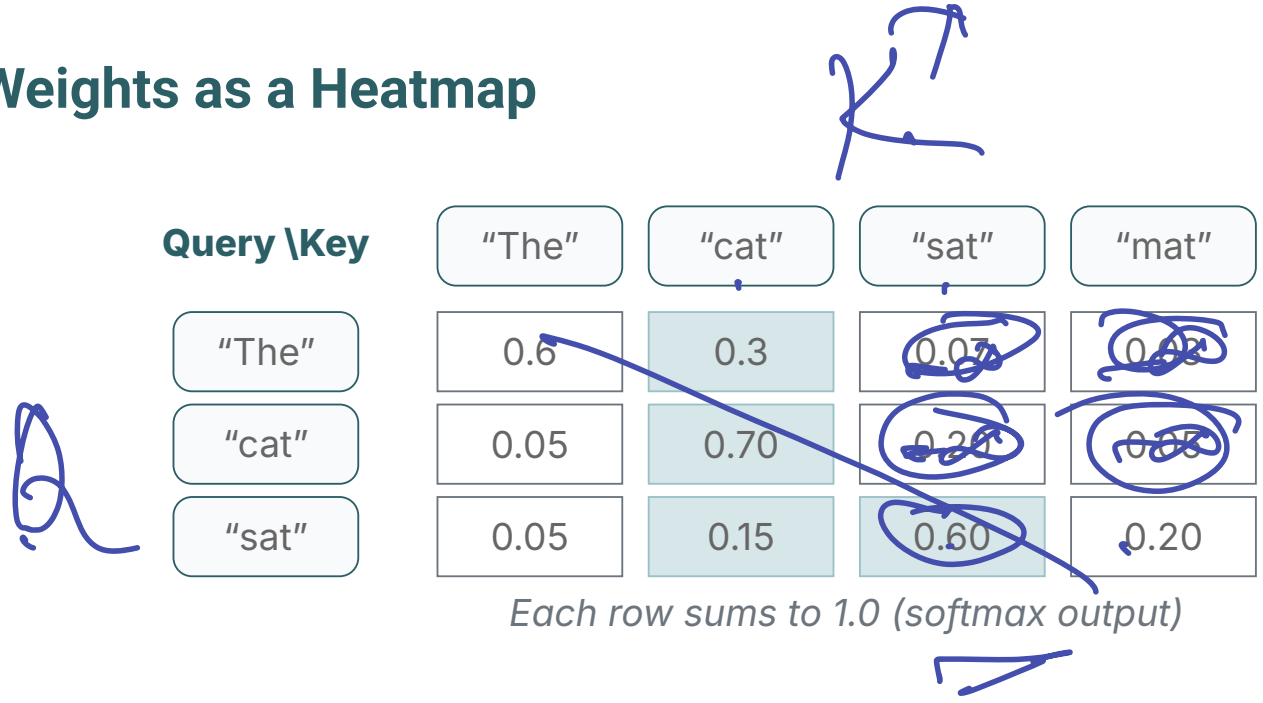
Why Scale by $\sqrt{d_k}$?

The Vanishing Gradient Problem Without Scaling

- For d_k-dimensional random unit vectors q and k, the dot product $q \cdot k$ has variance d_k (variance scales with dimensionality)
- For large d_k (e.g. $d_k = 64$), dot products have magnitude ~ 8 — softmax of values this large saturates to near 0/1 distributions
- Saturated softmax produces near-zero gradients, making attention weights untrainable in early training
- Dividing by $\sqrt{d_k}$ normalises the variance back to 1.0, keeping softmax in its well-gradient-behaved region

This single operation is the difference between stable training and gradient collapse.

Attention Weights as a Heatmap



Query \Key	"The"	"cat"	"sat"	"mat"
"The"	0.6	0.3	0.07	0.03
"cat"	0.05	0.70	0.20	0.05
"sat"	0.05	0.15	0.60	0.20

Each row sums to 1.0 (softmax output)

Worked Example: 2-Token Sequence, $d_k = 2$

Tracing Through the Formula by Hand

Setup: Two tokens, query/key dimension $d_k = 2$, value dimension $d_v = 2$

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad V = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

Step 1: $QK^\top = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ **Step 2:** Divide by $\sqrt{2} \approx 1.41$: $\begin{bmatrix} 0.71 & 0 \\ 0 & 0.71 \end{bmatrix}$

Step 3: Row-wise softmax: $A = \begin{bmatrix} 0.67 & 0.33 \\ 0.33 & 0.67 \end{bmatrix}$

Step 4: Output $= AV = \begin{bmatrix} 1.66 & 2.66 \\ 2.34 & 3.34 \end{bmatrix}$ — each output is a weighted blend of value vectors

Causal Masking for Autoregressive Generation

Preventing Future Token Leakage

In decoder self-attention, token at position i must not attend to positions $j > i$. This is enforced by adding a mask M before the softmax:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V$$

where $M_{ij} = -\infty$ if $j > i$, else 0.

What This Achieves

- $e^{-\infty} = 0$, so masked positions receive zero attention weight
- Training can proceed on all positions in parallel (teacher forcing)
- At inference, generation proceeds one token at a time left-to-right

Self-Attention vs. Cross-Attention

Self-Attention

- Q, K, V all come from the **same** sequence
- Models relationships *within* one sequence
- Used in both encoder and decoder layers
- Example: “bank” attends to “river” in the same sentence

Cross-Attention

- Q from one sequence, K and V from **another**
- Models relationships *across* two sequences
- Used in encoder-decoder and multimodal models
- Example: decoder attends to encoded source sentence

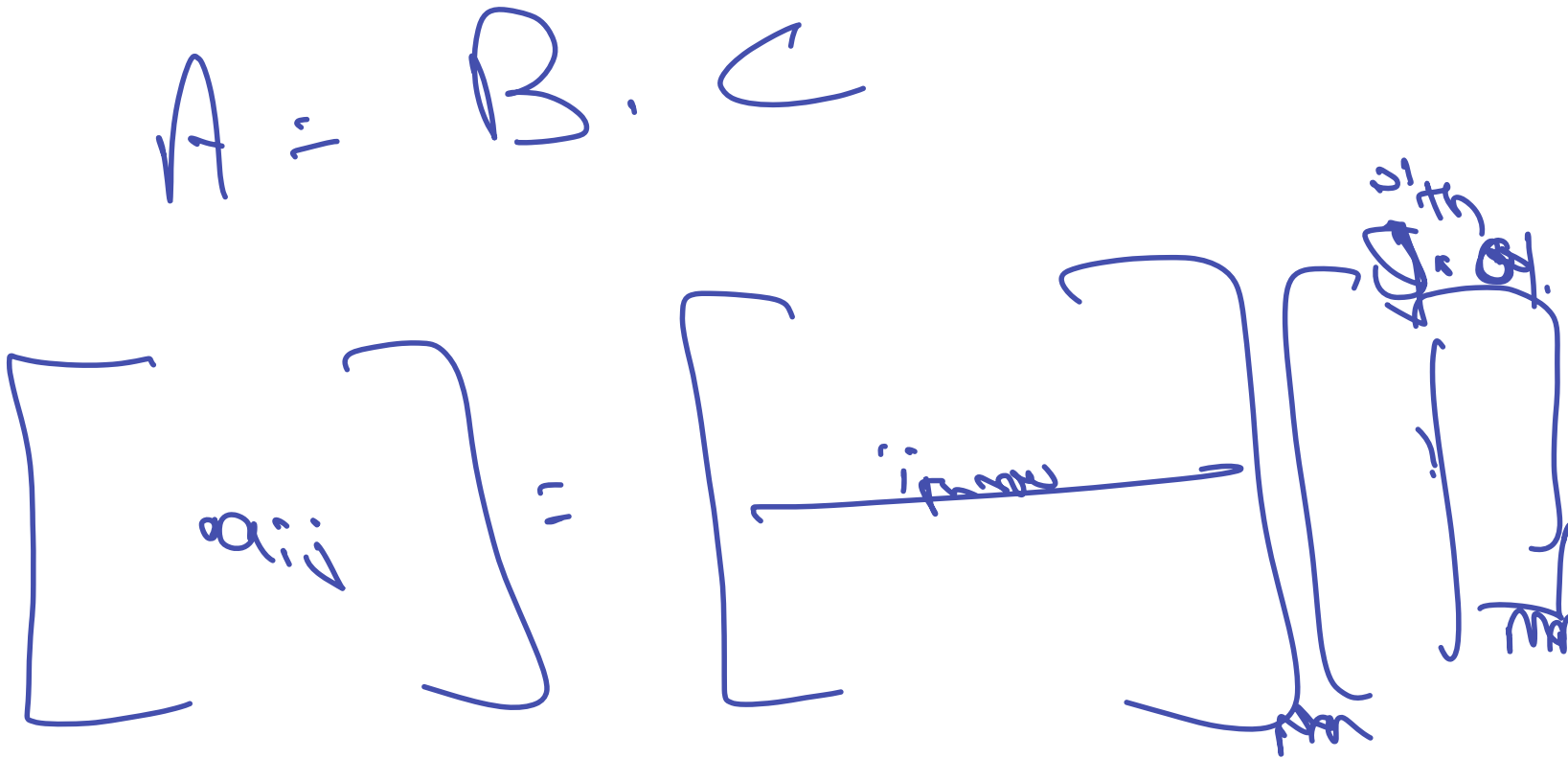
Computational Complexity

Time and Memory Complexity of Attention

- **Time:** $O(n^2 d_k)$ where n is sequence length. Computing $QK^\top$ is $O(n^2)$ — quadratic in sequence length
- **Memory:** The attention matrix A is $n \times n$, requiring $O(n^2)$ memory — the primary bottleneck for long-context models
- **For $n = 100{,}000$ tokens:** 10^{10} operations and 10 billion floats in the attention matrix — impractical without optimisation

Solutions in Modern Practice

Flash Attention (Dao et al. 2022): tiled GPU computation reduces memory to $O(n)$ while maintaining exact outputs. Now standard in PyTorch and HuggingFace.



GenAI Application: Retrieval-Augmented Generation

Cross-Attention Between Query and Retrieved Context

- **RAG pipeline:** User query → vector search → retrieved chunks are concatenated into the context window alongside the query
- **Attention role:** The LLM uses attention to identify which parts of the retrieved documents are most relevant to the query tokens
- **Perplexity AI:** Combines web search with a Transformer reading retrieved pages; attention over long contexts extracts relevant facts
- **Bing Copilot / Google AI Overviews:** Attention mechanism grounds generated answers in retrieved web content, reducing hallucination

The attention formula is the mechanism that connects retrieved knowledge to the generated response.

GenAI Application: Long-Context Language Models

Scaling Attention to Million-Token Contexts

- **Gemini 1.5 Pro (1M tokens):** Uses multi-query attention and linear attention approximations to make $O(n^2)$ tractable at extreme lengths
- **Claude 3 (200K tokens):** Combines Flash Attention with sparse attention patterns to handle book-length documents in a single pass
- **LongLoRA:** Extends pre-trained LLMs to longer contexts using shifted sparse attention — attention restricted to local windows plus global tokens
- **Jamba (AI21 Labs):** Hybrid Transformer-Mamba model; alternates attention layers (exact but $O(n^2)$) with SSM layers ($O(n)$) for efficiency

GenAI Application: Attention in Image Generation

Cross-Attention Links Text to Image Patches

- **Stable Diffusion / SDXL:** The denoising UNet uses cross-attention between text token embeddings (K, V) and image patch features (Q); this is how text descriptions control spatial content
- **DALL-E 3:** GPT-based text encoder + cross-attention to diffusion decoder enables detailed prompt following with spatial precision
- **DiT (Diffusion Transformer, Peebles et al. 2023):** Replaces UNet with a pure Transformer denoiser using self-attention on latent patches; basis for Sora (video generation) and Flux image generation
- **IP-Adapter:** Adds image-conditioned cross-attention to existing diffusion models for style transfer without full fine-tuning

GenAI Application: Attention in Code Understanding

Self-Attention for Syntactic and Semantic Structure in Code

- **GitHub Copilot / Codex:** Attention heads in code models learn to track variable scope, function signatures, and bracket matching across long code files — relationships impossible for RNNs to maintain
- **Tree-sitter + LLMs:** Syntactic parse trees are embedded as structured attention biases, improving attention alignment with code ASTs
- **AlphaCode 2:** Uses cross-attention between problem statement tokens and generated code tokens to ensure the solution matches the specification
- **Code review tools (CodeRabbit):** Attention identifies which lines of a diff relate to which issue descriptions in the pull request

Common Misconceptions

Clearing Up Confusion About Attention

- **"Attention weights tell us what the model is thinking":** Attention weights show what positions influenced each output — they are not the same as importance or semantic salience. Gradient-based methods are more reliable for interpretability.
- **"Larger attention scores mean more important tokens":** Scores before softmax are unbounded; only the softmax-normalised weights (which sum to 1) have meaningful magnitude comparisons.
- **"Self-attention is $O(n)$ like an RNN":** Self-attention is $O(n^2)$ in both time and memory. RNNs are $O(n)$ in memory but $O(n)$ sequentially — very different tradeoffs.

Summary

Scaled Dot-Product Attention — Key Takeaways

1. The formula $\text{softmax}(QK^\top/\sqrt{d_k})V$ computes a weighted sum of value vectors, where weights reflect query-key similarity
2. Scaling by $\sqrt{d_k}$ prevents softmax saturation and gradient vanishing when key dimensions are large
3. Causal masking adds $-\infty$ to future positions, enabling parallel training while preserving autoregressive generation at inference
4. Self-attention draws Q, K, V from the same sequence; cross-attention draws Q from one sequence and K/V from another
5. The $O(n^2)$ complexity is the central engineering challenge; Flash Attention solves memory but not time complexity

What This Concept Unlocks

Next in Module 3.2

- Encoder-Decoder Architecture
- Decoder-Only Architecture
- Layer Normalisation
- Feed-Forward Networks in Transformers
- Full Transformer Implementation (coding)

Later in the Course

- Flash Attention & Memory-Efficient Attention
- KV-Cache and Inference Optimisation
- Grouped-Query Attention (GQA)
- Multi-Query Attention (MQA)

Thank You

Scaled Dot-Product Attention | Module 3.2 | AhamX Bodhi